

遙測與空間資訊之分析與應用 - Week 6

國家災害防救科技中心 資訊組

台灣大學土木工程學系 教授

蘇文瑞

wraysu@ntu.edu.tw

The Captain's Intuition: Predicting the Unknown

- 🕒 **Week 5 Recap: We can monitor locations WITH sensors.**
- 🕒 **Week 6 Mission: What about the 95% of land with NO station?**
- 🕒 **Three Paths to Fill the Gaps:**
 - 🕒 Deterministic: IDW — nearest things win.
 - 🕒 Statistical: Kriging — spatial correlation.
 - 🕒 Machine Learning: Random Forest — data patterns.
- 🕒 **Vibe Coding prompt:**
 - 🕒 "I have 100 points, but I need a continuous field."
- 🕒 **Notebook Cell [1]: Load Typhoon Fung-wong rainfall station data.**

Spatial Interpolation: The Kriging Logic

- 🕒 Problem: Disasters don't only happen where sensors are.
- 🕒 Solution: Kriging — a method that fills in the blanks.
- 🕒 **Core idea: nearby stations should give similar readings.**
 - 🕒 A station 5 km away tells you more than one 50 km away.
 - 🕒 Kriging figures out how much “more” and uses that to predict.
- 🕒 **Disaster Scenario:**
 - 🕒 Typhoon Fung-wong: mountain stations failed.
 - 🕒 Kriging reconstructs the missing rainfall field.
 - 🕒 Predict landslide potential in unmonitored valleys.
- 🕒 **Think of it like weather forecasters coloring between dots on a map.**

Variogram: How Far Does Influence Travel?

- 🕒 **Question: How far apart can two stations be and still be related?**
- 🕒 Variogram — a chart that answers this:
 - 🕒 X-axis: Distance between station pairs (km).
 - 🕒 Y-axis: How different their rainfall readings are.
- 🕒 **Three numbers to read from it:**
 - 🕒 Nugget: The noise floor (measurement error).
 - 🕒 Sill: The maximum difference you expect.
 - 🕒 Range: Beyond this distance, stations don't help each other.
- 🕒 **Captain' s Audit:**
 - 🕒 If the dots don't follow the curve, your prediction is a lie.
- 🕒 Notebook Cell [2a]: First attempt — run it and see what happens.

Why Did It Fail? Look at the Histogram

- 🕒 **Before blaming the tool, look at your data.**
- 🕒 Plot a histogram of rainfall values:
 - 🕒 Most stations: 0–10 mm/hr (low rain).
 - 🕒 A few stations: 50–130 mm/hr (extreme rain).
- 🕒 **The problem: extreme values dominate everything.**
 - 🕒 The variogram tries to measure “typical” difference,
 - 🕒 but a few extreme stations make every pair look different.
- 🕒 **The fix: compress the extremes with log-transform.**
 - 🕒 $z_{\log} = \log(1 + z)$
 - 🕒 Now 130 mm becomes 4.9, and 1 mm becomes 0.7.
 - 🕒 The differences between stations become more balanced.
- 🕒 Notebook Cell [2b]: Histogram comparison.

The Fix: Log-Transform + Initial Parameters

- 🕒 **Step 1: Log-transform the data.**
 - 🕒 `z_log = np.log1p(z)`
 - 🕒 Compresses 130mm and 1mm into a similar scale.
- 🕒 **Step 2: Give the optimizer a reasonable starting point:**
 - 🕒 `sill = z_log.var()` (total variance after transform)
 - 🕒 `range = 50000` m (50 km, about 1/4 of study area)
 - 🕒 `nugget = sill * 0.1` (measurement error, ~10%)
- 🕒 **Step 3: After prediction, back-transform:**
 - 🕒 `z_pred = np.expm1(z_pred_log)`
 - 🕒 This converts log-scale predictions back to mm/hr.
- 🕒 **Captain' s Rule: Always check the variogram plot.**
- 🕒 Notebook Cell [2c]: Second attempt with log-transform.

Kriging Tuning: Why Extreme Rain Gets Diluted

- 🕒 **Problem: Suao=130mm, but Kriging predicts only 71mm at +500m. Why?**
- 🕒 Kriging is a weighted average — nearby low-rain stations dilute the peak.
- 🕒 **The #1 culprit: Nugget (measurement noise)**
 - 🕒 Nugget=10%: +500m → 71mm (46% dilution!)
 - 🕒 Nugget=1%: +500m → 107mm (only 18% dilution)
 - 🕒 CWA stations are calibrated → trust the readings, use low Nugget.
- 🕒 **Tune ONE variable at a time:**
 - 🕒 Step 1: Fix model (Spherical), vary Range → see Range effect.
 - 🕒 Step 2: Fix model (Exponential), vary Range → same exercise.
 - 🕒 Step 3: Fix Range, switch model → see Model effect.
 - 🕒 Smaller Range ≠ always better! It can increase dilution (see demo).
- 🕒 **For calibrated CWA stations: low Nugget + appropriate Range.**
- 🕒 Notebook Cells [2d] + [9b]: Variogram comparison + Nugget demo.

Toolstack: PyKriging & SciPy

- 🕒 PyKriging: the Python library that does Kriging for us.
- 🕒 **Three lines of code:**
 - 🕒 `from pykrige.ok import OrdinaryKriging`
 - 🕒 `OK = OrdinaryKriging(x, y, z_log, ...)`
 - 🕒 `z_pred, ss_pred = OK.execute('grid', grid_x, grid_y)`
- 🕒 Output: `z_pred` (predicted values) + `ss_pred` (confidence map).
- 🕒 **Always check the confidence map (`ss_pred`).**
- 🕒 ⚠️ **Important: `x, y` must be in meters (EPSG:3826).**
 - 🕒 Using lat/lon (degrees) gives wrong distances.
 - 🕒 1 degree $\approx$ 111 km, not 1 meter!
- 🕒 Notebook Cell [3]: Run Kriging and back-transform to mm/hr.

Machine Learning: A Different Approach

🕒 **Core Idea: Give the computer coordinates, let it learn the pattern.**

- 🕒 Input: station location (x, y) → Output: rainfall
- 🕒 The computer finds: “East side = more rain” by itself.

🕒 **Advantage: Easy to add more information.**

- 🕒 Input: (x, y, elevation, slope) → Output: rainfall
- 🕒 Terrain effects get learned automatically.

🕒 **Tool: Random Forest (scikit-learn)**

- 🕒 `rf = RandomForestRegressor(n_estimators=200)`
- 🕒 `rf.fit(X_train, y_train)`
- 🕒 `z_ml = rf.predict(X_grid)`

🕒 **Simpler code, but: no confidence map. Just “trust me.”**

🕒 Notebook Cell [4]: Random Forest prediction.

Kriging vs. ML: Which One to Trust?



Kriging:



Gives a confidence map (Sigma Map).



Based on how distance affects similarity.



Needs careful setup (variogram, log-transform).



Machine Learning (Random Forest):



Easy to add extra info (elevation, slope, etc.).



Simple code, fast results.



No confidence map. Just “here’s a number.”



The key difference:



Kriging tells you **WHERE** it’s guessing.



ML only tells you **WHAT** it thinks the answer is.



For disaster response: knowing WHERE you’re uncertain saves lives.

Peeking Inside the Black Box

🕒 Problem: “AI gave me a map, but I don’ t know why.”

🕒 **One way to peek: Feature Importance**

🕒 `rf.feature_importances_`

🕒 → [easting: 0.45, northing: 0.55]

🕒 Meaning: “The model relies more on north-south position.”

🕒 **Watch for: Minecraft Maps**

🕒 ML can memorize station values perfectly,

🕒 but between stations, it guesses badly.

🕒 Symptom: blocky, staircase-like artifacts.

🕒 **Captain’ s Audit:**

🕒 **“Does this map look like weather, or like a video game?”**

🕒 Notebook Cell [5]: Feature Importance analysis.

The Simplest Interpolators: NN & IDW

Before Kriging & ML, start with the two simplest spatial interpolators.

Nearest Neighbor (NN): Just copy the closest station.

- 10 Prediction at any point = the value of the nearest station.
- 10 Result: Voronoi tessellation (blocky map with sharp boundaries).
- 10 Fast & simple, but unrealistic jumps at cell boundaries.
- 10 scipy: `NearestNDInterpolator(points, values)`

Inverse Distance Weighting (IDW): Closer = more weight.

- 10 $w_i = 1 / d_i^p$ ($p=2$ is common; higher p = more local).
- 10 Weighted average of ALL stations, but far ones have tiny weight.
- 10 Smooth map, but no statistical basis & always underestimates peaks.
- 10 scipy: `cdist()` for distances, then manual weight calculation.

Why learn these? They are the baselines for Lab 1 (Slide 14).

- 10 If Kriging or ML can't beat NN/IDW, something is wrong with your model.

Four Methods Side by Side: Typhoon Fung-wong



Look at the 2×2 comparison plot:



Nearest Neighbor: patchwork quilt. Sharp ugly boundaries.



IDW: bullseye rings around each station.



Kriging: smooth, natural-looking surface.



Random Forest: blocky, Minecraft-like.



Why do they look different?



NN: Voronoi cells — each point copies its nearest station (diagonal boundaries).



IDW: $w = 1/d^p$ — concentric weight decay around each station.



Kriging: variogram-based weights — statistically optimal smoothing.



RF: decision trees split on if $x > \text{threshold}$ — axis-aligned cuts = blocky map.



The Shock Moment:



ML says: “80mm, trust me.”



Kriging says: “80mm, but I’m not sure about this area.”



You’ re the commander deciding whether to evacuate.



Who do you trust?



Notebook Cells [6]-[8]: Four-way comparison.

Lab 1: Prediction Shootout (40 min)

 **Task: Run Kriging and Random Forest on Typhoon Fung-wong data.**

-  Produce the 2×2 comparison plot.
-  Analyze physical intuition: which result looks more realistic?

 **Teacher Patrol Checklist:**

-  CRS = EPSG:3826? (not lat/lon)
-  RF features = [easting, northing]? (not latitude, longitude)
-  Variogram Sill $\neq$ 0? If Sill = 0, guide student to add variogram_parameters.
-  IDW using cdist with power=2? (not Rbf inverse)

 **Students have: Kriging API + RF API + four-method visualization.**

The Sigma Map: Kriging's Unique Weapon

- 🕒 **Kriging gives TWO maps, not one:**
 - 🕒 Map 1: Predicted rainfall (same as ML).
 - 🕒 Map 2: Confidence — how sure are we at each location?
- 🕒 **Reading the Confidence Map:**
 - 🕒 Blue (low) = near stations = reliable estimate.
 - 🕒 Dark (high) = far from stations = uncertain.
- 🕒 **For the Commander:**
 - 🕒 High rain + confident = EVACUATE NOW.
 - 🕒 High rain + uncertain = SEND MORE SENSORS FIRST.
- 🕒 **ML can't do this. It only gives you the first map.**
- 🕒 **"The Sigma Map is your honest report card."**
- 🕒 Notebook Cell [9]: Sigma Map visualization + Cell [9b]: Nugget demo.

GeoTIFF Export: Making Results Reusable



Why GeoTIFF?



QGIS / ArcGIS can open it directly.



Overlay with DEM, land use, infrastructure layers.



Preserves spatial reference (CRS + affine transform).



Output Files:



kriging_rainfall.tif



kriging_variance.tif



rf_rainfall.tif



⚠ Y-Axis Flip (Critical!):



NumPy row 0 = south, but GeoTIFF row 0 = north.



Always use `np.flipud(array)` before writing to rasterio.



Notebook Cell [10]: GeoTIFF export with rasterio.

One Pattern, Solve a Hundred Problems



The Week 6 workflow:



Get data → Predict surface (Kriging or ML) → Make a map



→ Overlay with buildings/roads → Decision for the Commander



This works for:



Earthquakes, landslides, air quality, epidemics, heat islands...



Any time you have scattered measurements and need a full picture.



When to use Kriging:



Few stations, need confidence, only coordinates available.



When to consider ML:



More data available (elevation, slope, land use, etc.).



Master this workflow, and you can handle any spatial prediction.

Lab 2: Confidence & Uncertainty (30 min)



Task:

-  Generate Kriging Sigma Map.
-  Export results as GeoTIFF (with `np.flipud!`).
-  Reflect: Why can't ML produce a similar confidence map?



Bonus:

-  Zonal Statistics by township using `rasterstats`.
-  AI Advisor consultation: ask Gemini to interpret your Sigma Map.



Teacher Patrol:

-  Variance map colormap choice (diverging vs sequential).
-  Student understanding of what high variance means for decisions.

Final Project Kickoff: The Captain's Mission



You now have the tools:



Vector / Raster / API / AI / Kriging / Machine Learning.



Now you need a MISSION.



Teams: Free grouping, 1–4 people.



Solo or small teams welcome.



Same deliverable expectations regardless of team size.



Suggested Roles (for multi-person teams):



Data Captain: API integration + data cleaning.



Spatial Architect: Kriging, Zonal Stats, overlay analysis.



AI UX Lead: Gemini SDK integration + Folium dashboard.



Topic Constraint: Taiwan-specific disaster scenario.

Technical Requirements

- 🕒 **Data Fusion:** Integrate at least 1 Vector + 1 Raster source.
- 🕒 **Spatial Analysis:** Must include Kriging, Zonal Stats, or advanced overlay.
- 🕒 **AI Integration:** Must use Gemini SDK for real-time decision advice.
- 🕒 **Open Data:** At least 1 open data source.
- 🕒 **Visualization:** Folium interactive map or analysis dashboard.
- 🕒 **Infrastructure: Complete GitHub repo.**
 - 🕒 Full commit history.
 - 🕒 .env for API key management (gitignored).
 - 🕒 Modular code in src/ directory.
 - 🕒 README.md with AI diagnostic log.

Project Theme Inspiration



Theme A: Urban Flash Flood Warning



CWA real-time rainfall + land use (Vector) + DEM (Raster).



Goal: Automated underground parking lot flood alert system.



Theme B: Mountain Landslide Alert



Kriging rainfall interpolation + slope analysis + road network.



Goal: Predict road closures and issue early warnings.



Theme C: Earthquake Impact Audit



Seismic intensity (Points) + building footprints (OSM) + AI logic.



Goal: Post-earthquake damage estimation dashboard.



Theme D: Extreme Heat / Heat Island



Temperature stations + land use + population density.



Goal: Identify high-risk neighborhoods for heat wave response.

Project Timeline & Milestones

-  **Week 6 (This Week): Team formation + topic selection.**
-  **Week 7: 1-page proposal outline due.**
 -  Team roster, research question, data sources, methods, Gemini SDK plan.
-  **Week 8–9: Midterm progress report (informal 5-min check-in).**
-  **Week 10–15: Regular class continues + project development in spare time.**
-  **Week 16: Final Showcase — The Captain's Showcase.**
 -  10-minute presentation + 5-minute Q&A.

-  **Peer Review Culture:**
 -  Today: each team presents their mission plan for feedback.
-  **"Critique is the fuel for better engineering."**

Vibe Coding for Projects + AI Advisor

🕒 **Don' t just prompt: "Analyze this."**

🕒 **Strategic Prompt:**

- 🕒 "Given my W4 slope model and W5 rainfall feed, write a function
- 🕒 to identify hotspots using Kriging."

🕒 **Gemini as Advisor:**

- 🕒 "Audit my project logic."
- 🕒 "Suggest the best Kriging model for this data density."
- 🕒 "Based on these 3 critical points, what should the commander do?"

🕒 **GitHub Repo Structure:**

- 🕒 project/ .env .gitignore src/ data/ outputs/
- 🕒 ARIA_final.ipynb README.md

Week 6 Reflection

- 🕒 "An estimate without uncertainty is just a guess.
- 🕒 **An estimate with uncertainty is science."**
- 🕒 Interpolation is not just filling space;
 - 🕒 it is predicting risk where sensors cannot reach.
- 🕒 Kriging' s Sigma Map is more valuable than
 - 🕒 a perfect-looking ML prediction with no confidence measure.
- 🕒 Method Choice:
 - 🕒 Sparse data + need confidence? → Kriging.
 - 🕒 Rich features + dense data? → Consider ML.
- 🕒 **Homework: Part A (Kriging vs ML) + Part B (Project Proposal).**
- 🕒 **Next: Start building your Final Project. The Commander is waiting.**